

Responde con tus propias palabras

¿Cuál fue el primer lenguaje de programación que empleaste?

El primer lenguaje de programación que empleé fue C.

¿Cuál es tu principal lenguaje de programación actualmente?

Actualmente mi principal lenguaje de programación es Java.

¿Cuál es la diferencia entre un package.json y composer.json?

- **package.json**

Administra las dependencias de Node y almacena metadatos relevantes del proyecto para NPM.

- **composer.json**

Es el archivo con el que Composer administra las dependencias de PHP.

¿Qué es

`/(<script(\s|S)*?</script>)(<style(\s|S)*?</style>)(<!--(\s|S)*?-->)(<\/?(\s|S)*?>)/gi?`

Es una expresión regular o regex.

¿Para qué sirve la orden 'docker-compose up'?

Sirve para compilar, (re)crear, iniciar, y añadir contenedores a un servicio.

¿Qué es el patrón MVC?

El patrón MVC (Modelo-Vista-Controlador) se define como un patrón de arquitectura de software que, mediante la utilización de sus tres elementos (modelo, vista y controlador), separa la lógica de negocio de la lógica de presentación.

¿Indica algún Framework MVC y qué aporta en el desarrollo de un proyecto?

Un ejemplo de Framework MVC es Kendo, utilizado para desarrollo web y móvil. Aporta componentes para UI web, UI mobile y visualización de datos, lo que acelera la construcción de interfaces y reduce trabajo repetitivo.

¿Qué es un ORM?

Es una práctica de desarrollo que transforma datos entre un lenguaje orientado a objetos y una base de datos relacional.

¿Indica algún ORM y qué aporta en el desarrollo de un proyecto?

Un ejemplo de ORM es ActiveRecord. Aporta una forma más simple de interactuar con la base de datos y reduce la complejidad del acceso a persistencia.

¿Qué es el patrón MVVM?

Es un patrón de arquitectura cuyo objetivo es desacoplar al máximo la lógica de la aplicación de la interfaz de usuario mediante los elementos modelo, vista y view model.

¿Indica algún Framework MVVM y qué aporta en el desarrollo de un proyecto?

Un ejemplo de Framework MVVM es MvvmCross. Sus aportaciones principales son su potencia y su comunidad activa, lo que facilita evolución y soporte.

¿Para qué sirve la orden 'git stash'?

git stash aparta temporalmente los cambios desde el último commit y deja limpio el directorio de trabajo para poder cambiar de contexto y recuperarlos más adelante.

¿Indica qué problemas identificas en el siguiente código?

```
public function indexAction(Application $app, Request $request)
{
    if ($app->getConfig('maintenance')
        && !in_array($request->getClientIp(),
            $app->getConfig('excludeIpsFromMaintenance'))) {
        return $app['twig']->render(
            '@MainLayouts/maintenance.html.twig',
            ['endDate' => $app->getConfig('maintenanceEndDate')]
        );
    }
    try {
        $databaseUser = $app->getDatabaseUser();
        $user = new User($databaseUser);
        if (!$databaseUser->getProfile()->isInitialized()) {
            $preferences = $this->getApp()->getEm()
                ->getRepository('\CS\Entity\OcPreferences')
                ->findOneBy(['userid' => $databaseUser->getUid(),
                    'configkey' => 'lang', 'appid' => 'core']);
            if (null != $preferences) {
                $form = $user->getInitForm(
                    $databaseUser->getProfile()->getDisplayname(),
                    $preferences->getConfigvalue());
            }
        }
        return $app['twig']->render('@MainLayouts/init.html.twig',
            ['form' => $form->createView()]);
    } catch (\Exception $e) {
        throw $e;
    }
}
```

- El catch es redundante porque solo vuelve a lanzar la misma excepción sin aportar contexto ni tratamiento adicional.
- throw \$e; añade ruido y es peor que dejar que la excepción burbujee sin capturarla.
- \$form puede quedar sin inicializar si el perfil ya está inicializado o si findOneBy(...) devuelve null, y aun así después se usa en \$form->createView().
- getProfile() se invoca varias veces sin comprobar antes si devuelve null, por lo que podría romper al llamar a isInitialized() o getDisplayname().
- La acción mezcla dos formas de acceder a dependencias, usando a la vez \$app y \$this->getApp(), lo que introduce inconsistencia y mayor acoplamiento.
- Se hace una consulta a persistencia directamente desde la acción, mezclando lógica de controlador con acceso a datos y dificultando pruebas y mantenimiento.
- El if (null != \$preferences) \$form = ... sin llaves reduce claridad y facilita errores en futuras modificaciones.
- in_array(...) se usa sin comparación estricta, lo que no es ideal.
- Se asume que excludeIpsFromMaintenance siempre devuelve un array válido; si no lo hace, in_array(...) puede fallar.

Pruebas técnicas

1) Implementa en php empleando clases, herencia e interfaces atendiendo a los principios SOLID una solución que proporcione aquellos números pertenecientes a una sucesión de Fibonacci que:

- Se encuentre entre el timestamp del inicio y fin del mes actual.
- Se encuentre entre el timestamp del inicio y fin del año actual.
- Se encuentre entre el timestamp del inicio y fin entre dos fechas atendiendo al formato "Y-m-d H:i:s".

El timezone a emplear para las fechas es el UTC. La solución ha de funcionar sobre CLI. Se ha de entregar el ejercicio en una carpeta llamada PHP.

2) Implementa en ECMAScript v6+ empleando clases y herencia atendiendo a los principios SOLID una solución que proporcione aquellos números pertenecientes a una sucesión de Fibonacci que:

- Se encuentre entre el timestamp del inicio y fin del mes actual.
- Se encuentre entre el timestamp del inicio y fin del año actual.
- Se encuentre entre el timestamp del inicio y fin entre dos fechas atendiendo al formato "Y-m-d H:i:s".

La implementación ha de realizarse con Vanilla JS. Se han de poder agregar las fechas desde un formulario web y mostrar los resultados de los tres casos indicados en la misma vista del formulario. Se ha de entregar el ejercicio en una carpeta llamada JAVASCRIPT.

3) Implementa un formulario de login con HTML5 y Bootstrap. Con los siguientes campos y sus correspondientes validadores:

- Usuario: este ha de ser un correo electrónico.
- Contraseña: ha de tener un mínimo de 8 caracteres, al menos 1 letra mayúscula, 1 letra minúscula y no puede contener caracteres especiales.
- Un checkbox de recordar contraseña.
- Un botón de submit.
- Ha de llamar por POST a la url <https://dinahosting.com/login-usuarios>.

El formulario no ha de emplear javascript y se valorará el empleo de preprocesadores CSS. Se ha de entregar el ejercicio en una carpeta llamada CSS.